

# C3M4\_Peer\_Reviewed

August 13, 2024

```
[138]: library(ggplot2)
       library(mgcv)
```

## 1 C3M4 Peer Reviewed Assignment

### 1.0.1 Outline:

The objectives for this assignment:

1. Observe the difference between GAMs and other regression models on simulated data.
2. Review how to plot and interpret the coefficient linearity for GAM models.

General tips:

1. Read the questions carefully to understand what is being asked.
2. This work will be reviewed by another human, so make sure that you are clear and concise in what your explanations and answers.

```
[139]: # Load packages
       library(ggplot2)
       library(mgcv)
```

## 2 Problem 1: GAMs with Simulated Data

In this example, we show how to check the validity of a generalized additive model (GAM) (using the `gam()` function) using simulated data. This allows us to try and understand the intricacies of `gam()` without having to worry about the context of the data.

### 2.0.1 1. (a) Simulate the Data

Let  $n = 200$ . First, construct three predictor variables. The goal here is to construct a GAM with different types of predictor terms (e.g., factors, continuous variables, some that will enter linearly/parametrically, some that enter transformed).

1. `x1`: A continuous predictor that, we will suppose has a nonlinear relationship with the response.
2. `x2`: A categorical variable with three levels: `s`, `m`, and `t`.
3. `x3`: A categorical variable with two levels: `TRUE` and `FALSE`.

Then, make the response some nonlinear/nonparametric function of  $\mathbf{x}$ . For our case, use:

$$\log(\mu_i) = \beta_1 + \sin(0.5x_{i,1}^2) - x_{i,2} + x_{i,3}$$

This model is a Poisson GAM. In a realworld situation, we wouldn't know this functional relationship and would estimate it. Other terms are modeled parametrically. The response has normal noise.

Note that:

1. The construction of  $\boldsymbol{\mu} = (\mu_1, \dots, \mu_n)^T$  has the linear predictor exponentiated, because of the nature of the link function.
2. We use  $\boldsymbol{\mu}$  to construct  $\mathbf{y} = (y_1, \dots, y_n)^T$ . The assumption for Poisson regression is that the random variable  $Y_i$  that generates  $y_i$  is Poisson with mean  $\mu_i$ .
3. `as.integer(as.factor(VARIABLE))` converts the labels of VARIABLE to 1, 2, 3,.. so that we can construct the relationship for these factors.

Plot the relationship of  $\mathbf{y}$  to each of the predictors. **Then, split the data into a training (`train_sim`) and test (`test_sim`) set.**

```
[189]: set.seed(0112358)
# n = number of data points
n = 200

x1 = runif(n, -pi/2, pi/2)
x2 = as.factor(sample(c("s", "m", "t"), 200, replace=TRUE))
x3 = as.factor(sample(c(TRUE, FALSE), 200, replace=TRUE))

mu = exp(sin(0.5*x1^2)-as.integer(x2)+as.integer(x3))

y = rpois(n, mu);

simul.data = data.frame(x1=x1, x2=x2, x3=x3, y =y)
head(simul.data)

m = floor(0.8 * nrow(simul.data))
index = sample(seq_len(nrow(simul.data)), size = m)

train_sim = simul.data[index, ]
test_sim = simul.data[-index, ]
```

|   | x1<br><dbl> | x2<br><fct> | x3<br><fct> | y<br><int> |
|---|-------------|-------------|-------------|------------|
| 1 | -0.5693977  | t           | FALSE       | 0          |
| 2 | 0.7552628   | m           | FALSE       | 1          |
| 3 | 1.1752773   | t           | TRUE        | 1          |
| 4 | 0.5986983   | s           | FALSE       | 1          |
| 5 | 0.6344793   | m           | TRUE        | 4          |
| 6 | 0.9273050   | t           | FALSE       | 1          |

## 2.0.2 1. (b) Other Regression Models

Before jumping straight into GAMs, let's test if other regression models work. What about a regular linear regression model with ordinary least squares, and a generalized linear model for Poisson regression?

First fit a linear regression model to your `train_sim` data. We know that all of the predictors were used to make the response, but are they all significant in the linear regression model? Explain why this may be.

Then fit a Generalized Linear Model (GLM) to the `train_sim` data. Plot three diagnostic plots for your GLM:

1. Residual vs. log(Fitted Values)
2. QQPLOT of the Residuals
3. Actual Values vs. Fitted Values

Using these plots, determine whether this model is a good fit for the data. Make sure to explain your conclusions and reasoning.

```
[190]: # Fit a LM model to the data
lm.mod <- lm(y~x1+x2+x3, data=train_sim)
summary(lm.mod)

# Fit a GLM model to the data
glm.mod <- glm(y~x1+x2+x3, data=train_sim, family=poisson(link="log"))
names(summary(glm.mod))

#residual plot
ggplot(data.frame(fitted=glm.mod$fitted.values, deviance_resid=summary(glm.
  mod)$deviance.resid), aes(y=deviance_resid,
  x=fitted))+geom_point()+geom_hline(yintercept=0)
ggplot(glm.mod, aes(y=.resid, x=log(
  fitted)))+geom_point()+geom_hline(yintercept=0)

# Create the three specified plots
summary(glm.mod)
pchisq(glm.mod$deviance, glm.mod$df.residual, lower.tail=FALSE)

## qqplot
```

```
qqnorm(glm.mod$residuals)
qqline(glm.mod$residuals)

#fitted vs actual
ggplot(data.frame(fitted=glm.mod$fitted, actual=train_sim$y), aes(x=actual,
→y=fitted))+geom_point()+geom_abline()
```

Call:

```
lm(formula = y ~ x1 + x2 + x3, data = train_sim)
```

Residuals:

| Min     | 1Q      | Median | 3Q     | Max    |
|---------|---------|--------|--------|--------|
| -2.8113 | -0.7015 | 0.1415 | 0.4225 | 4.1995 |

Coefficients:

|             | Estimate | Std. Error | t value | Pr(> t )     |
|-------------|----------|------------|---------|--------------|
| (Intercept) | 2.66800  | 0.19685    | 13.554  | < 2e-16 ***  |
| x1          | -0.09298 | 0.09601    | -0.968  | 0.334        |
| x2s         | -2.05738 | 0.23779    | -8.652  | 6.05e-15 *** |
| x2t         | -2.82909 | 0.22298    | -12.687 | < 2e-16 ***  |
| x3TRUE      | 0.99603  | 0.18052    | 5.518   | 1.41e-07 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.14 on 155 degrees of freedom

Multiple R-squared: 0.5635, Adjusted R-squared: 0.5522

F-statistic: 50.03 on 4 and 155 DF, p-value: < 2.2e-16

1. 'call' 2. 'terms' 3. 'family' 4. 'deviance' 5. 'aic' 6. 'contrasts' 7. 'df.residual' 8. 'null.deviance'  
 9. 'df.null' 10. 'iter' 11. 'deviance.resid' 12. 'coefficients' 13. 'aliased' 14. 'dispersion' 15. 'df'  
 16. 'cov.unscaled' 17. 'cov.scaled'

Warning message in log(.fitted):

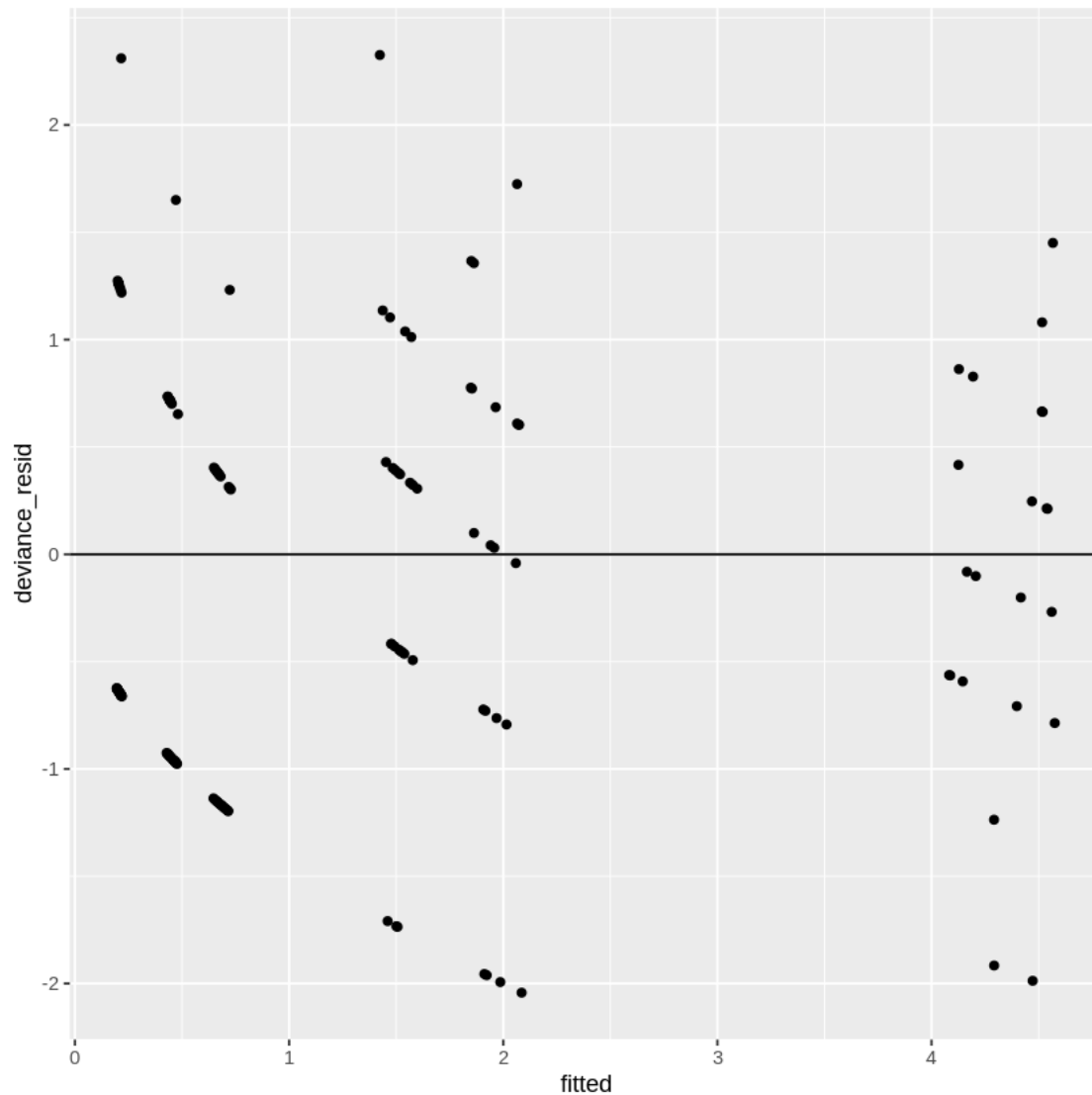
“NaNs produced”

Warning message in log(.fitted):

“NaNs produced”

Warning message:

“Removed 93 rows containing missing values (geom\_point).”



Call:

```
glm(formula = y ~ x1 + x2 + x3, family = poisson(link = "log"),
    data = train_sim)
```

Deviance Residuals:

| Min     | 1Q      | Median  | 3Q     | Max    |
|---------|---------|---------|--------|--------|
| -2.0425 | -0.9405 | -0.5279 | 0.6030 | 2.3258 |

Coefficients:

|             | Estimate | Std. Error | z value | Pr(> z )     |
|-------------|----------|------------|---------|--------------|
| (Intercept) | 0.67498  | 0.13436    | 5.024   | 5.07e-07 *** |
| x1          | -0.03907 | 0.06945    | -0.562  | 0.574        |
| x2s         | -1.04971 | 0.16144    | -6.502  | 7.91e-11 *** |

```

x2t      -2.25116    0.23003   -9.786   < 2e-16 ***
x3TRUE    0.78650    0.14894    5.281  1.29e-07 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

(Dispersion parameter for poisson family taken to be 1)

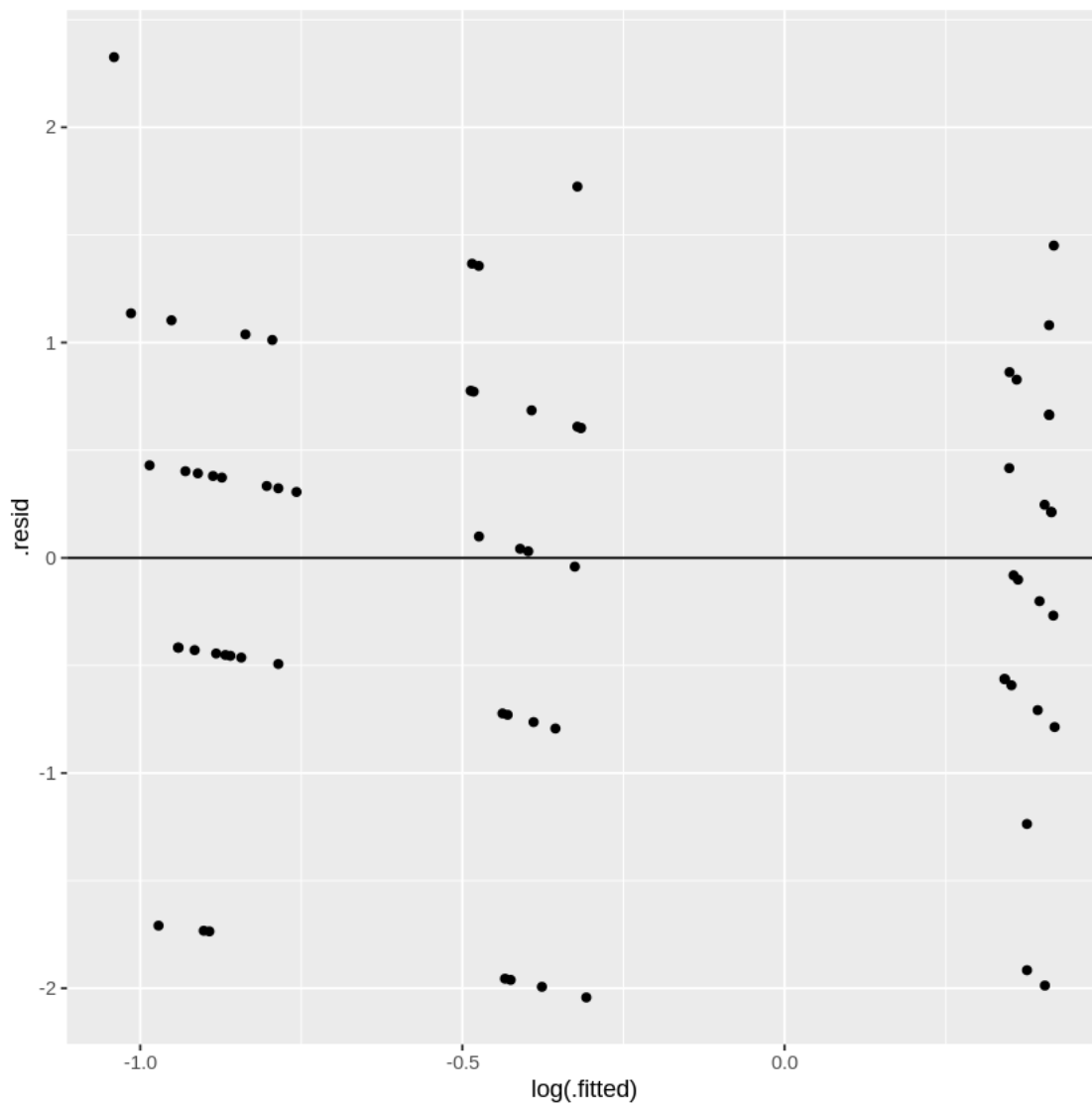
```

Null deviance: 330.98  on 159  degrees of freedom
Residual deviance: 144.64  on 155  degrees of freedom
AIC: 390.19

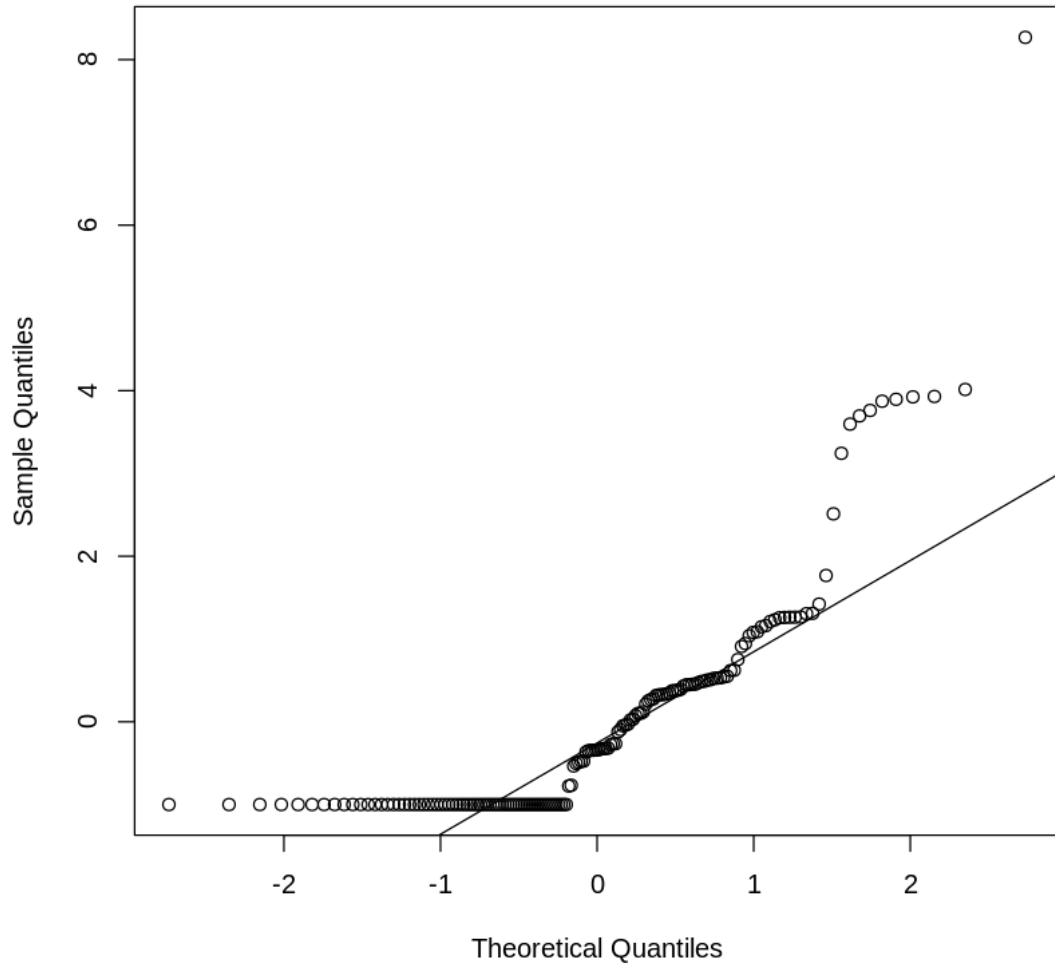
```

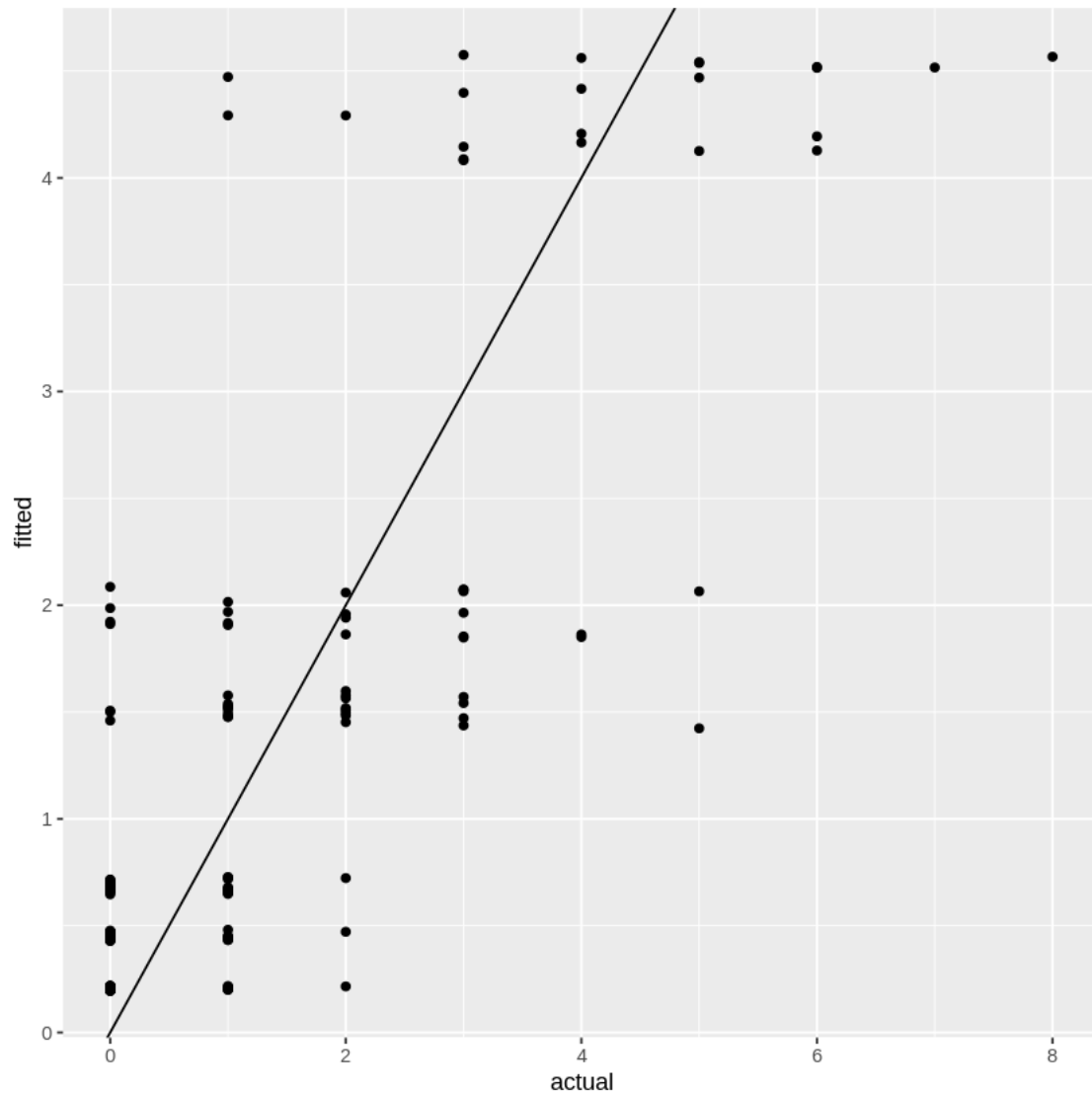
Number of Fisher Scoring iterations: 5

0.713562427906398



Normal Q-Q Plot





## 2.1 Answers

1. **LM INTERPRETATION** - all parameters in the `lm.mod` are significant, except for `x1`. This is obvious because the response variable was constructed with a non-linear relation to `x1`. Thus, the response does not vary significantly linearly with `x1`
2. **deviance\_residual vs fitted plot** - The points are not randomly scattered about the horizontal line.
3. **qqplot** - The qqplot is useless here - the response variable is poisson distributed.
4. **fitted vs actual** - The fitted vs actual show the increase in dispersion as the values increase.

Thus, the model seems to be a bad fit for the data.



## 2.2 \*\*\*\*\*

### 2.2.1 1. (c) Looking for those GAMs

Now, it's time to see how a generalized additive model (GAM) performs! Fit a GAM to the data. Construct the same three plots for your GAM model. Do these plots look better than those of the GLM?

```
[193]: # Fit a GAM model to the data
g = gam(y ~ s(x1) + x2 + x3, data = train_sim, family = poisson)
summary(g)
names(g)
names(summary(g))
# Construct the three specified plots

#residual vs fitted
ggplot(data.frame(residuals=g$residuals, fitted=log(g$fitted.values)),
  aes(y=residuals, x=fitted))+geom_point()+geom_hline(yintercept=0)

## qqplot
qqnorm(g$residuals)
qqline(g$residuals)

#fitted vs actual
ggplot(data.frame(fitted=g$fitted, actual=train_sim$y), aes(x=actual,
  y=fitted))+geom_point()+geom_abline()
```

Family: poisson

Link function: log

Formula:

$y \sim s(x1) + x2 + x3$

Parametric coefficients:

|             | Estimate | Std. Error | z value | Pr(> z ) |     |
|-------------|----------|------------|---------|----------|-----|
| (Intercept) | 0.5714   | 0.1416     | 4.035   | 5.47e-05 | *** |
| x2s         | -0.9309  | 0.1663     | -5.598  | 2.16e-08 | *** |
| x2t         | -2.1581  | 0.2326     | -9.277  | < 2e-16  | *** |
| x3TRUE      | 0.8130   | 0.1500     | 5.421   | 5.93e-08 | *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

|       | edf   | Ref.df | Chi.sq | p-value  |
|-------|-------|--------|--------|----------|
| s(x1) | 2.367 | 2.943  | 9.556  | 0.0224 * |

---

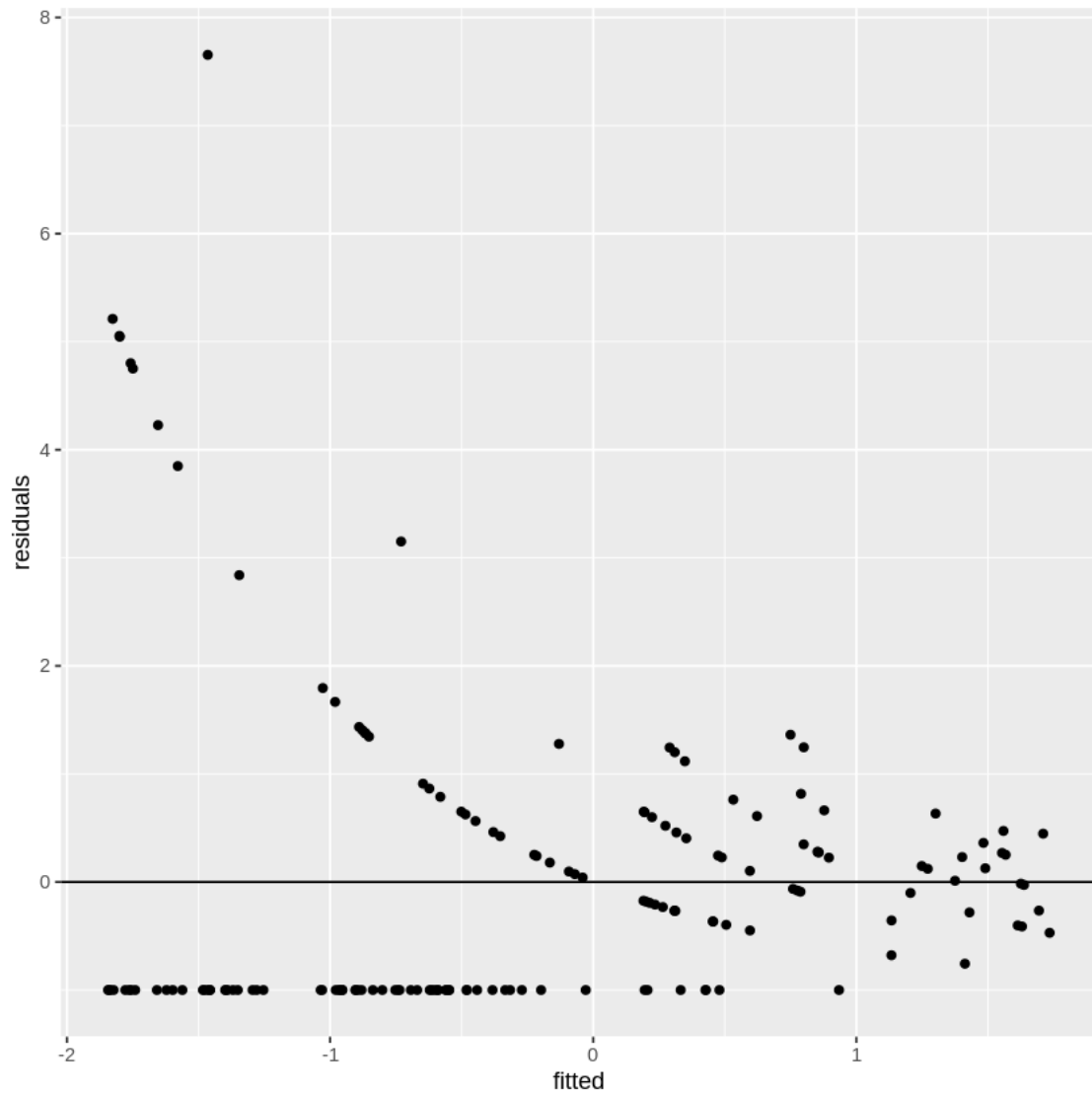
Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

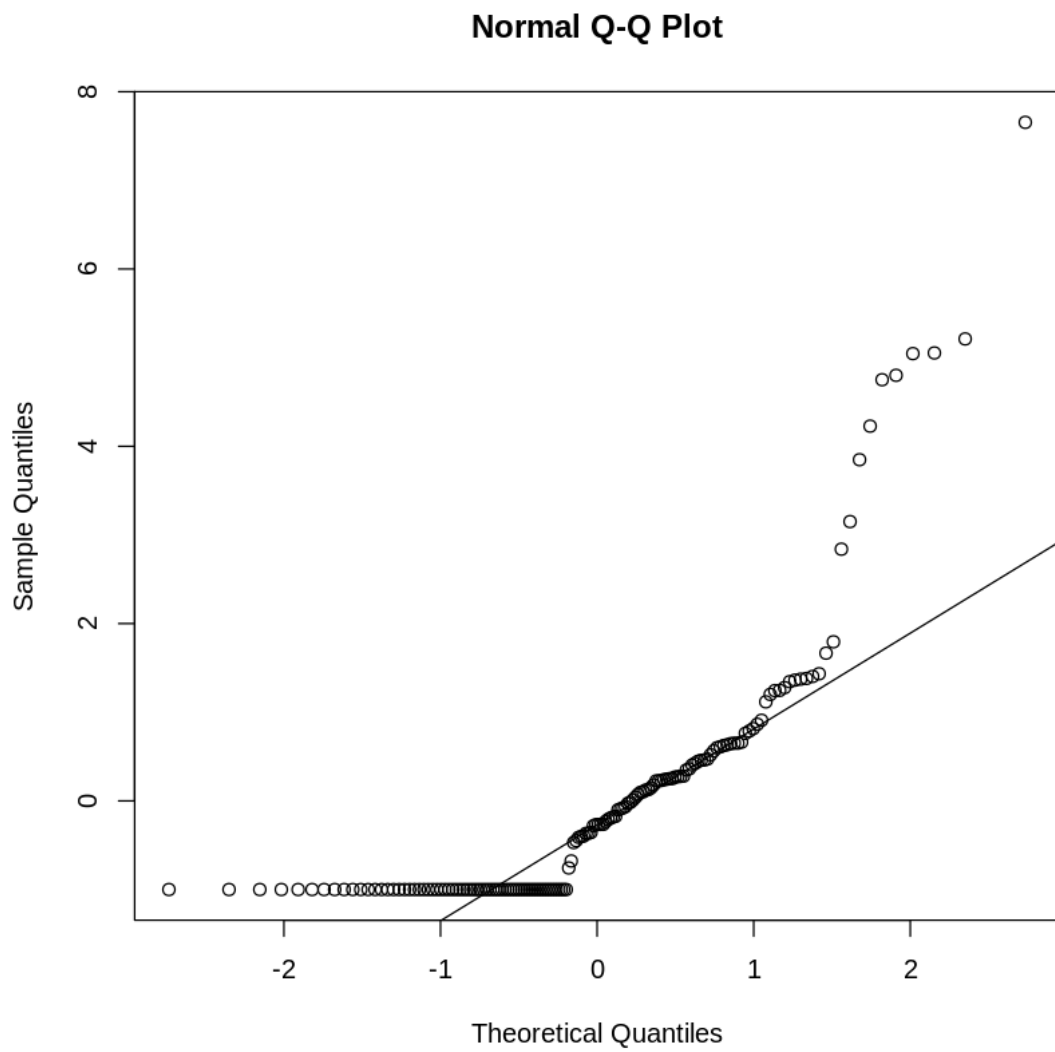
R-sq.(adj) = 0.673 Deviance explained = 59.8%

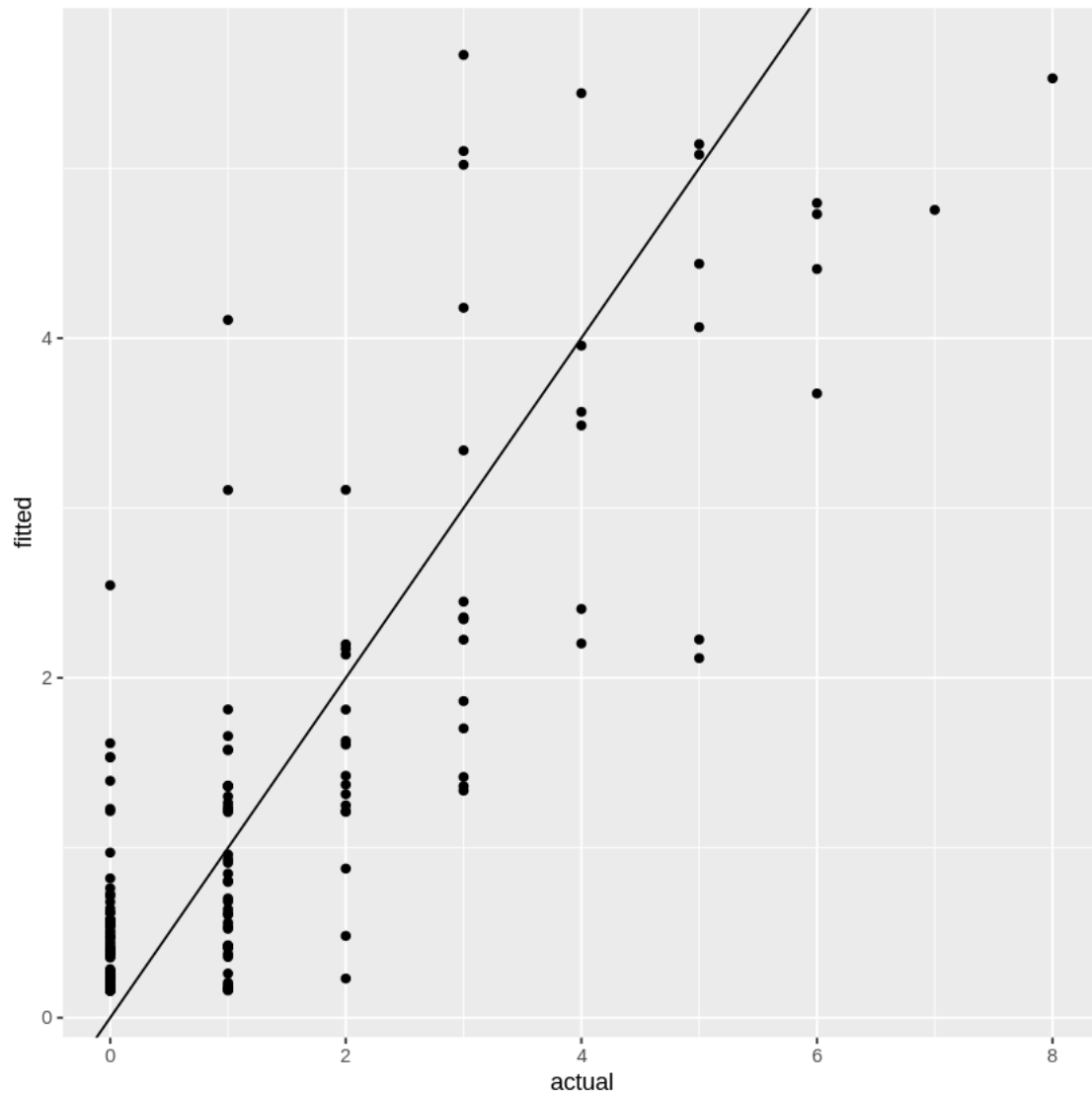
UBRE = -0.088726 Scale est. = 1 n = 160

1. 'coefficients' 2. 'residuals' 3. 'fitted.values' 4. 'family' 5. 'linear.predictors' 6. 'deviance'  
7. 'null.deviance' 8. 'iter' 9. 'weights' 10. 'working.weights' 11. 'prior.weights' 12. 'df.null' 13. 'y'  
14. 'converged' 15. 'boundary' 16. 'rV' 17. 'db.drho' 18. 'dw.drho' 19. 'ldetS1' 20. 'reml.scale'  
21. 'aic' 22. 'rank' 23. 'sp' 24. 'gcv.ubre' 25. 'outer.info' 26. 'scale.estimated' 27. 'scale' 28. 'con-  
trol' 29. 'method' 30. 'Vp' 31. 'hat' 32. 'Ve' 33. 'edf' 34. 'edf1' 35. 'R' 36. 'nsdf' 37. 'sig2' 38. 'smooth'  
39. 'formula' 40. 'var.summary' 41. 'cmX' 42. 'model' 43. 'terms' 44. 'pred.formula' 45. 'pterm'  
46. 'assign' 47. 'contrasts' 48. 'xlevels' 49. 'offset' 50. 'df.residual' 51. 'min.edf' 52. 'optimizer'  
53. 'call'

1. 'p.coeff' 2. 'se' 3. 'p.t' 4. 'p.pv' 5. 'residual.df' 6. 'm' 7. 'chi.sq' 8. 's.pv' 9. 'scale' 10. 'r.sq' 11. 'fam-  
ily' 12. 'formula' 13. 'n' 14. 'dev.expl' 15. 'edf' 16. 'dispersion' 17. 'pTerms.pv' 18. 'pTerms.chi.sq'  
19. 'pTerms.df' 20. 'cov.unscaled' 21. 'cov.scaled' 22. 'p.table' 23. 'pTerms.table' 24. 's.table'  
25. 'method' 26. 'sp.criterion' 27. 'rank' 28. 'np'







## 2.3 Answers

The residuals vs fitted shows that the variance decreases with increasing values.

The qqplot does not matter.

The fitted values don't track the actual values at all, which is sign of a bad fit.

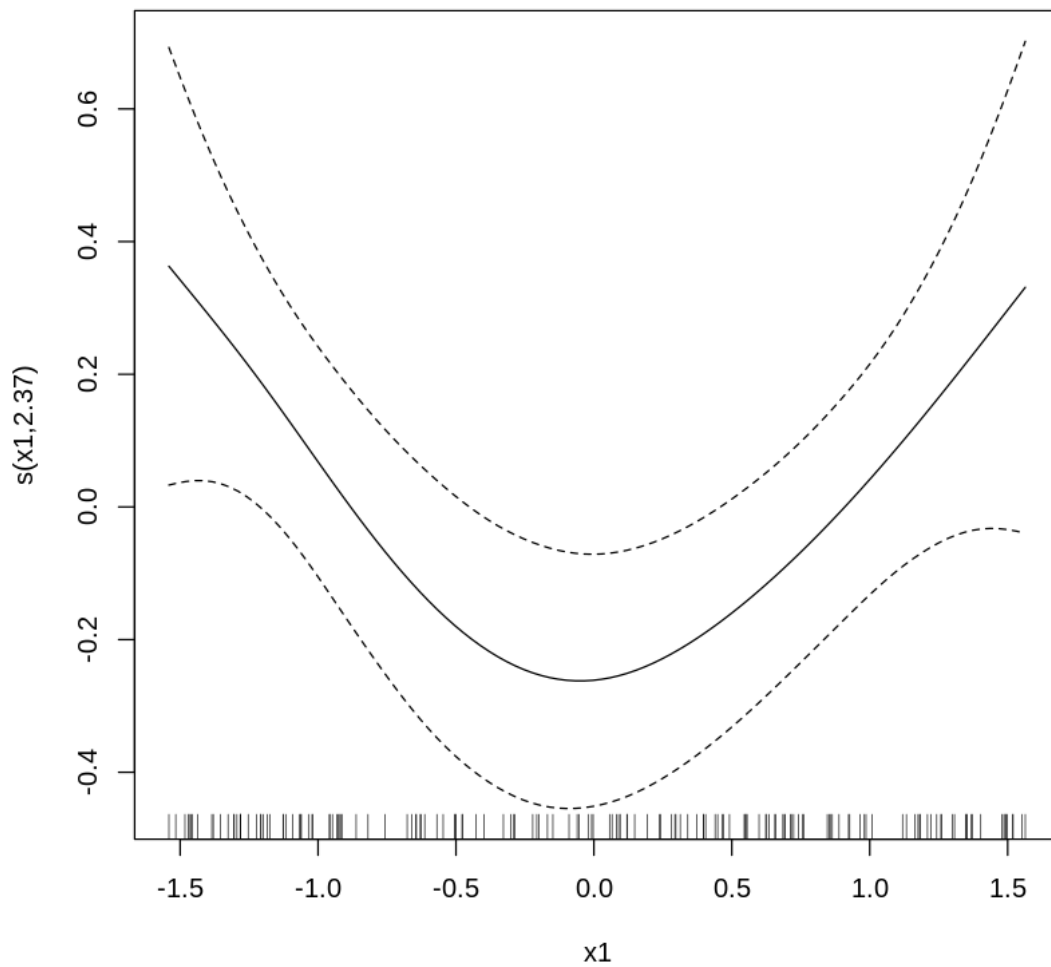
## 2.4 \*\*\*\*\*

### 2.4.1 1. (d) Interpreting GAMs

We made a GAM model! However GAMs are harder to interpret than regular linear regression models. How do we determine if a GAM model was necessary? Or, in other words, how do we determine if our predictors have a linear relationship with the response?

Use the `plot.gam()` function in the `mgcv` library to plot the relationship between  $y$  and  $x_1$ . Recall that  $x_1$  entered our model as  $\sin(0.5x_{i,1}^2)$ , and we plotted that relationship in **1.(a)**. Does your plot confirm this relationship?

```
[194]: plot.gam(g)
```



## 2.5 Answers

Yes, it is squigly enough. It is not possible to draw a straight between the confidence intervals.

## 2.6 \*\*\*\*\*

### 2.6.1 1.(e) Model comparison

Compute the mean squared prediction error (MSPE) for each of the three models above (regression model, GLM, and GAM). State which model performs based according to this metric.

Remember, the MSPE is given by

$$MSPE = \frac{1}{k} \sum_{i=1}^k (y_i^* - \hat{y}_i^*)^2$$

where  $y_i^*$  are the observed response values in the test set and  $\hat{y}_i^*$  are the predicted values for the test set (using the model fit on the training set).

```
[195]: #mspe for lm
mean((test_sim$y-exp(predict(lm.mod, test_sim)))^2)

#mspe for glm
mean((test_sim$y-exp(predict(glm.mod, test_sim)))^2)

# mspe for gam
mean((test_sim$y-exp(predict(g, test_sim)))^2)
```

386.729260957177

1.47619259831457

0.866262681206378

## 2.7 Answers

GAM is the best.

## 2.8 \*\*\*\*\*

## 3 Problem 2 Additive models with the advertising data

The following dataset contains measurements related to the impact of three advertising medias on sales of a product,  $P$ . The variables are:

- youtube: the advertising budget allocated to YouTube. Measured in thousands of dollars;
- facebook: the advertising budget allocated to Facebook. Measured in thousands of dollars; and
- newspaper: the advertising budget allocated to a local newspaper. Measured in thousands of dollars.
- sales: the value in the  $i^{th}$  row of the sales column is a measurement of the sales (in thousands of units) for product  $P$  for company  $i$ .

The advertising data treat “a company selling product  $P$ ” as the statistical unit, and “all companies selling product  $P$ ” as the population. We assume that the  $n = 200$  companies in the dataset were chosen at random from the population (a strong assumption!).

First, we load the data, plot it, and split it into a training set (`train_marketing`) and a test set (`test_marketing`).

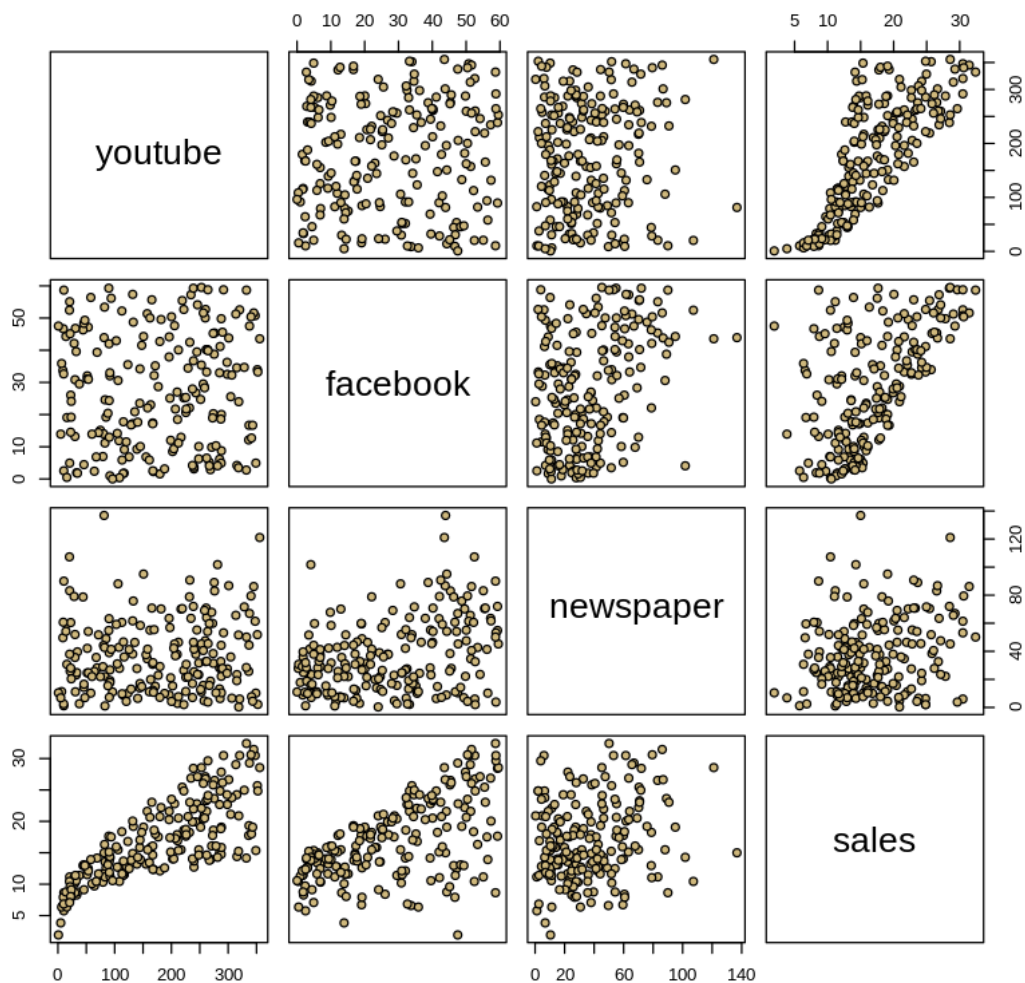
```
[196]: # Load in the data
marketing = read.csv("marketing.txt", sep = "")
head(marketing)
pairs(marketing, main = "Marketing Data", pch = 21,
      bg = c("#CFB87C"))
```

A data.frame: 6 × 4

|   | youtube<br><dbl> | facebook<br><dbl> | newspaper<br><dbl> | sales<br><dbl> |
|---|------------------|-------------------|--------------------|----------------|
| 1 | 276.12           | 45.36             | 83.04              | 26.52          |
| 2 | 53.40            | 47.16             | 54.12              | 12.48          |
| 3 | 20.64            | 55.08             | 83.16              | 11.16          |
| 4 | 181.80           | 49.56             | 70.20              | 22.20          |
| 5 | 216.96           | 12.96             | 70.08              | 15.48          |
| 6 | 10.44            | 58.68             | 90.00              | 8.64           |



## Marketing Data



```
[201]: set.seed(177) #set the random number generator seed.
n = floor(0.8 * nrow(marketing)) #find the number corresponding to 80% of the
  ↳data
index = sample(seq_len(nrow(marketing)), size = n) #randomly sample indices to
  ↳be included in the training set

train_marketing = marketing[index, ] #set the training set to be the randomly
  ↳sampled rows of the dataframe
test_marketing = marketing[-index, ] #set the testing set to be the remaining
  ↳rows

dim(test_marketing) #check the dimensions
dim(train_marketing) #check the dimensions
```

1. 40 2. 4

1. 160 2. 4

## 2.(a) Let's try a GAM on the marketing data!

Note that the relationship between sales and youtube is nonlinear. This was a problem for us back in the first course in this specialization, when we modeled the data as if it were linear. In the last module, we focused on modeling the relationship between sales and youtube, omitting the other variables. Now it's time to include the additional predictors.

Using the `train_marketing` fit an additive model to the data and store it in `gam_marketing`. Produce the relevant added variable plots using `plot(gam_marketing)`. Comment on the fit of the model.

```
[202]: gam_marketing <- gam(sales~s(youtube)+facebook+newspaper, data=train_marketing)
summary(gam_marketing)
plot.gam(gam_marketing)
```

Family: gaussian

Link function: identity

Formula:

sales ~ s(youtube) + facebook + newspaper

Parametric coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t )   |
|-------------|-----------|------------|---------|------------|
| (Intercept) | 11.264013 | 0.279555   | 40.293  | <2e-16 *** |
| facebook    | 0.186582  | 0.008530   | 21.874  | <2e-16 *** |
| newspaper   | 0.004403  | 0.005811   | 0.758   | 0.45       |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

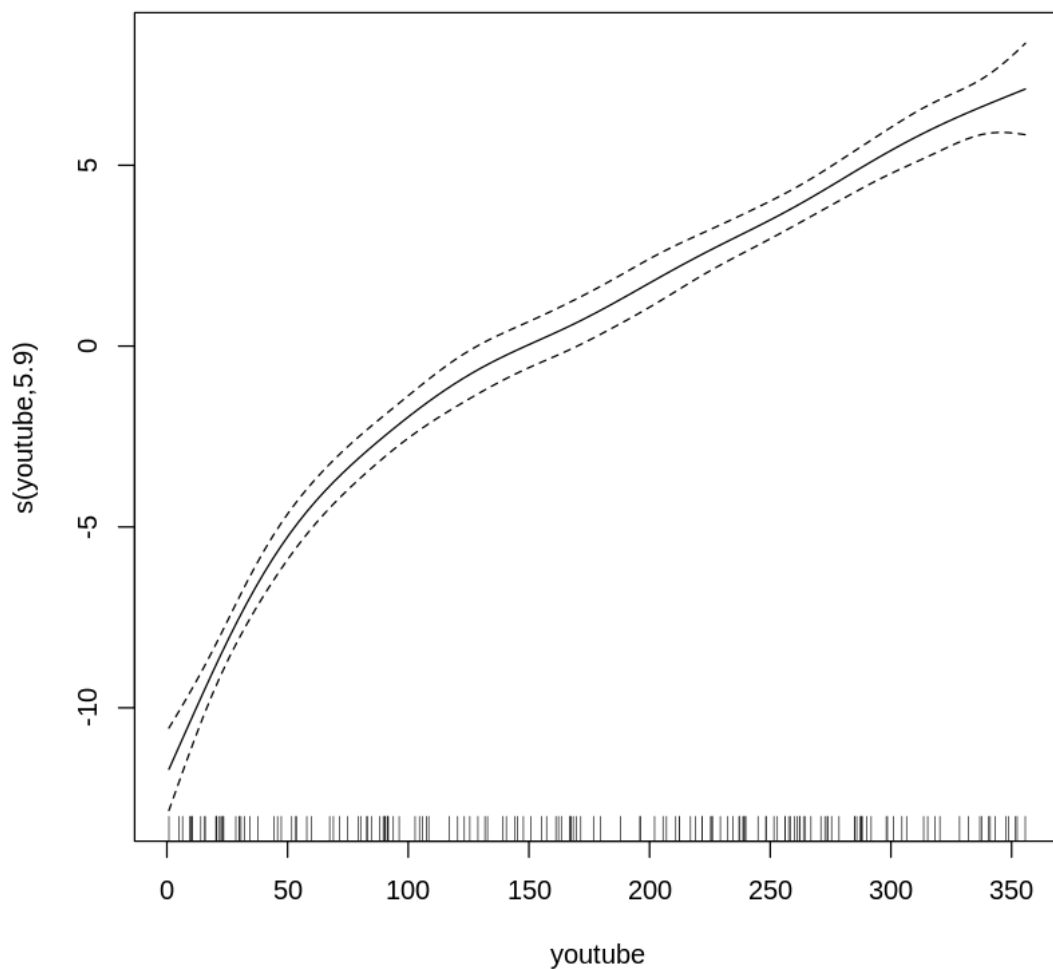
|            | edf | Ref.df | F     | p-value    |
|------------|-----|--------|-------|------------|
| s(youtube) | 5.9 | 7.047  | 202.1 | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.929 Deviance explained = 93.2%

GCV = 2.9718 Scale est. = 2.8065 n = 160



### 3.1 Answers

93.2% of the deviance is explained so the models does fit well

The relation between youtube and sales is non linear. There is no way to fit a linear relation between the two givent the narrow confidence intervals. The high edf and p-value~0 further support this conclusion.

### 3.2 \*\*\*\*\*

#### 2.(b) Semiparametric modeling of the marketing data

Refit the additive model based on your results from 2.(a). That is, if any predictors above should enter linearly, refit the model to reflect that. If any predictors are statistically insignificant, remove

them from the model. Store your final model in `semiparametric_marketing`.

```
[203]: semiparametric_marketing <- gam(sales~s(youtube)+facebook, data=train_marketing)
summary(semiparametric_marketing)
```

Family: gaussian

Link function: identity

Formula:

`sales ~ s(youtube) + facebook`

Parametric coefficients:

|             | Estimate  | Std. Error | t value | Pr(> t )   |
|-------------|-----------|------------|---------|------------|
| (Intercept) | 11.352099 | 0.253895   | 44.71   | <2e-16 *** |
| facebook    | 0.189085  | 0.007848   | 24.09   | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

|            | edf   | Ref.df | F     | p-value    |
|------------|-------|--------|-------|------------|
| s(youtube) | 5.979 | 7.127  | 200.9 | <2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.929 Deviance explained = 93.2%

GCV = 2.9438 Scale est. = 2.797 n = 160

### 3.3 Answers

The newspaper was removed due to high p-value.

### 3.4 \*\*\*\*\*

#### 2.(c) Model comparisons

Now, let's do some model comparisons on the test data. Compute the mean squared prediction error (MSPE) on the `test_marketing` data for the following three models:

- `gam_marketing` from 2.(a)
- `semiparametric_marketing` from 2.(b)
- `lm_marketing`, a linear regression model with `sales` is the response and `youtube` and `facebook` are predictors (fit on the `train_marketing` data).

State which model performs based according to this metric.

```
[204]: #mspe for gam
mean((test_marketing$sales-predict(gam_marketing, test_marketing))^2)
```

```
#mspe for semiparametric
mean((test_marketing$sales-predict(semiparametric_marketing, test_marketing))^2)

# mspe for lm
lm.mod <- lm(sales~youtube+facebook, data=train_marketing)
mean((test_marketing$sales-predict(lm.mod, test_marketing))^2)
```

3.40994916862721

3.29744826352071

4.19770116857387

### 3.5 Answers

semiparametric seems to be the best.

### 3.6 \*\*\*\*\*

[ ]: